EKSPLORASI PERFORMA DAN SKALABILITAS MACHINE LEARNING PADA MAHADATA MENGGUNAKAN APACHE SPARK (SPARK MLLIB)

Replikasi Parsial Komparasi Model Klasifikasi Terdistribusi Berdasarkan Studi Kasus Higgs Boson



Nama : Hanifah Hasanah

NIM : 123140082

Mata Kuliah : Mahadata / Big Data (IF25-40402)

Dosen Pengampu : Meida Cahyo Untoro, S.Kom., M.Kom.

Tautan Repositori GitHub:

https://github.com/HANIFAHHASANAH-123140082/123140082_Hanifah_Hasanah_Eksplorasi_Mahadata_Spark.git

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNOLOGI INDUSTRI

INSTITUT TEKNOLOGI SUMATERA

2026

A. RINGKASAN JURNAL ILMIAH RUJUKAN

1. Identitas dan Sitasi Artikel Rujukan

- **Judul Artikel** : Applying Machine Learning on Big Data With Apache Spark
- **Penulis** : Elias Dritsas dan Maria Trigka
- **Afiliasi Penulis** : Department of Informatics and Computer Engineering, University of West Attica, Athena, Yunani
- **Nama Jurnal** : IEEE Access (Multidisciplinary Open Access Journal)
- **Tahun Terbit / Volume** : Tahun 2025, Volume 13, Halaman 53377-53393
- **Tanggal Publikasi** : Diterima 12 Maret 2025, Dipublikasikan 17 Maret 2025
- **Nomor DOI** : 10.1109/ACCESS.2025.3552042
- **Sitasi** : Dritsas, E., & Trigka, M. (2025). Applying machine learning on big data with Apache Spark. IEEE Access, 13, 53377-53393.

2. Masalah Mahadata yang Diselesaikan

Pertumbuhan data digital berskala masif memicu kendala *bottleneck* latensi I/O pada sistem analitik tradisional berbasis piringan magnetik (*disk-based*) ketika mengeksekusi algoritma *Machine Learning* (ML) yang bersifat iteratif. Penelitian Dritsas dan Trigka (2025) bertujuan menguji efisiensi komputasi *in-memory* Apache Spark untuk menjembatani implementasi teoritis model ML dengan skala operasional mahadata nyata. Fokus penelitian mencakup evaluasi performa prediktif, analisis skalabilitas waktu pelatihan dan inferensi, serta pemantauan konsumsi sumber daya perangkat keras fisik (*CPU utilization* dan konsumsi RAM).

3. Dataset dan Lingkungan Komputasi Artikel Asli

Penelitian rujukan menguji tiga dataset publik berukuran raksasa:

1. **NYC Taxi Trip Duration:** 100 juta rekaman untuk kasus regresi durasi perjalanan.
2. **Netflix Prize:** 100 juta ulasan film dari 480.000 pengguna untuk sistem rekomendasi dan regresi.
3. **Higgs Boson:** 11 juta rekaman tumbukan partikel fisika energi tinggi untuk klasifikasi biner, terdiri atas 28 fitur numerik (*kinematic features*) dengan rasio ketimpangan kelas 65% sampel non-partikel (*background*, kelas 0) dan 35% sampel partikel (*signal*, kelas 1).

Lingkungan Komputasi: Klaster Apache Spark 8 node *worker* (setiap node memiliki 8 core CPU dan 64 GB RAM, total kapasitas 64 core CPU dan 512 GB RAM) berbasis Apache Hadoop YARN dan HDFS dengan Apache Spark versi 3.0.1.

4. Metode dan Arsitektur Komputasi yang Digunakan

Alur kerja komputasi artikel memanfaatkan pustaka *Spark MLlib* di atas DataFrame terdistribusi:

- **Pra-pemrosesan Data** : Standardisasi fitur *Z-score* dan *one-hot encoding*
- **Partisi Data** : Rasio 80% data latih dan 20% data uji.
- **Model yang Diuji** : Kasus regresi (*Linear Regression*, RF, GBT, SVR, KNN) dan klasifikasi (*Logistic Regression*, RF, GBT, SVM, KNN).
- **Optimasi Hyperparameter** : *Grid Search* dengan 5-*fold cross-validation* paralel. Metrik evaluasi mencakup Akurasi, *F1-Score*, AUC, durasi waktu komputasi, serta utilitas CPU dan RAM.

5. Hasil Utama Artikel Rujukan

Temuan eksperimental artikel Dritsas dan Trigka (2025) menghasilkan sejumlah wawasan ilmiah penting:

1. **Skalabilitas Waktu Linear** : Peningkatan volume data (skala 10% hingga 100%) menghasilkan peningkatan waktu komputasi pelatihan dan prediksi yang proporsional linier berkat kemampuan partisi data Spark.
2. **Keunggulan Ensemble**: Pada klasifikasi dataset Higgs Boson 11 juta data penuh, model pohon gabungan mendominasi metrik evaluasi. GBT meraih akurasi 90,5% (AUC 0,97) dan Random Forest meraih akurasi 90,0% (AUC 0,96), melampaui Logistic Regression (87,5%, AUC 0,94).
3. **Efisiensi Sumber Daya Model Linier**: Model linier (LR dan LinR) mencatatkan waktu pelatihan tersingkat (LR membutuhkan 29 menit pada 100% data, sedangkan RF membutuhkan 140 menit) serta konsumsi CPU (35%–55%) dan RAM (13%–42%) terendah di seluruh pengujian.

6. Alasan Ilmiah Memilih Artikel Rujukan

Artikel ini relevan dengan materi mata kuliah Mahadata, terbit di jurnal internasional bereputasi tinggi (*IEEE Access* Q1, 2025), serta metodologi klasifikasi biner Spark MLlib

pada studi kasus Higgs Boson sangat layak untuk direplikasi secara parsial pada lingkungan komputasi mandiri.

B. TAHAPAN IMPLEMENTASI DAN EKSPLORASI MANDIRI

1. Tingkat Replikasi yang Dipilih

Tingkat replikasi yang dipilih adalah Replikasi Parsial (*Partial Replication*). Replikasi berfokus pada klasifikasi biner dataset Higgs Boson. Mengingat dataset asli berukuran 11 juta baris pada klaster 8 node, eksperimen mandiri menggunakan sampel representatif 50.000 baris data dengan tetap mempertahankan struktur 28 fitur kinematik dan proporsi ketimpangan kelas asli (65% kelas 0 dan 35% kelas 1) untuk membandingkan *Logistic Regression* dan *Random Forest*.

2. Lingkungan dan Versi Perangkat Lunak

Eksperimen dijalankan pada lingkungan *notebook* komputasi awan dengan spesifikasi berikut:

- Platform eksekusi: Google Colaboratory (Linux x86_64, Ubuntu)
- Bahasa pemrograman: Python 3.13 dan Apache Spark versi 4.0.4 (*local mode*)
- Alokasi driver: *spark.driver.memory = 4g*
- Pustaka utama: PySpark 4.0.4, Py4J 0.10.9.9, NumPy 1.26.4, Matplotlib 3.10.0. Seluruh dependensi dicatat dalam berkas requirements.txt pada repositori GitHub.

3. Rancangan dan Alur Kerja Eksperimen

Eksperimen terdiri atas tahapan berikut, mengacu pada alur kerja Spark MLlib dalam artikel rujukan:

1. Inisialisasi SparkSession (memori driver 4 GB)
2. Pembentukan dan pembacaan DataFrame (50.000 baris, 28 fitur, skema StructType)
3. Transformasi fitur menjadi vektor dengan VectorAssembler
4. Pembagian data: 80% latih dan 20% uji (seed 42)
5. Pelatihan dua model: Logistic Regression dan Random Forest
6. Pengukuran waktu komputasi dan evaluasi metrik (Akurasi, F1-Score, AUC)
7. Visualisasi dengan Matplotlib dan analisis perbandingan

4. Langkah-Langkah Teknis Implementasi

- **Langkah 1:** Inisialisasi Apache Spark Session.

  Sesi Spark lokal dibangun dengan memori driver 4 GB untuk mencegah galat kehabisan memori (*Java Out of Memory Error*) selama pelatihan model:

```python
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Eksplorasi-Higgs-SparkML") \
    .config("spark.driver.memory", "4g") \
    .getOrCreate()
```

- **Langkah 2:** Pembentukan Dataset Sintetis.

  Untuk meniru karakteristik dataset Higgs Boson (UCI *Machine Learning Repository*) yang digunakan Dritsas dan Trigka (2025), dibuat dataset sintetis 50.000 sampel menggunakan NumPy dengan *random seed* 42:

  - Label target (y): Biner (0.0 atau 1.0) dengan distribusi probabilitas p = [0,65; 0,35], mencerminkan rasio 65% *background* dan 35% *signal*.
  - Fitur prediktor (X): Matriks 28 kolom numerik kontinu (*float/double*) yang merepresentasikan sifat kinematik partikel.
  - Data digabung menjadi 29 kolom dan disimpan sebagai higgs_sample.csv.

- **Langkah 3:** Pembacaan Data dan Pembentukan Skema Spark DataFrame.

  Berkas CSV dimuat ke dalam *DataFrame* Apache Spark dengan mendefinisikan skema kolom secara eksplisit (*StructType* dan *DoubleType*) untuk menghindari *overhead* komputasi akibat inferensi skema otomatis (*inferSchema*):

```python
from pyspark.sql.types import StructType, StructField, DoubleType

schema_fields = [StructField("label", DoubleType(), True)] + [
    StructField(f"f{i}", DoubleType(), True) for i in range(1, 29)
]
schema = StructType(schema_fields)

df = spark.read.csv("higgs_sample.csv", schema=schema, header=False)
```

- **Langkah 4**: Rekayasa Fitur dengan VectorAssembler.

  Pustaka Spark MLlib mensyaratkan seluruh fitur prediktor digabungkan ke dalam satu kolom bertipe data vektor. Transformasi dilakukan menggunakan VectorAssembler yang memetakan kolom f1 sampai f28 menjadi satu kolom vektor tunggal bernama features, kemudian memilih kolom features dan label sebagai masukan pemodelan:

```python
from pyspark.ml.feature import VectorAssembler

feature_cols = [f"f{i}" for i in range(1, 29)]
assembler = VectorAssembler(inputCols=feature_cols, outputCol="features")
final_data = assembler.transform(df).select("features", "label")
```

- **Langkah 5**: Partisi Data Latih dan Data Uji.

  Sesuai metodologi artikel rujukan, dataset dibagi dengan proporsi 80% untuk data latih (*training set*) dan 20% untuk data uji (testing set) menggunakan fungsi randomSplit dengan seed 42. Hasil partisi menghasilkan:

  - Data Latih: 40.104 baris data terdistribusi.
  - Data Uji: 9.896 baris data terdistribusi.

- **Langkah 6**: Konfigurasi Parameter dan Pelatihan Model.

  1. *Logistic Regression* (LR): featuresCol="features", labelCol="label", maxIter=20. Algoritma meminimalkan fungsi kerugian logistik (*cross-entropy loss*) secara iteratif.
  2. *Random Forest* (RF): featuresCol="features", labelCol="label", numTrees=20, maxDepth=5, seed=42. Model membangun 20 pohon keputusan dengan kedalaman maksimal 5 level.

- **Langkah 7**: Pengukuran Waktu dan Evaluasi Metrik.

  Selama fase pelatihan dan pengujian, modul pustaka time digunakan untuk mencatat:

  - Waktu Latih (Training Time): Durasi waktu eksekusi saat metode .fit(train_data) dijalankan.

- Waktu Prediksi (*Testing Time / Inference*): Durasi waktu eksekusi saat metode .transform(test_data) menghasilkan label prediksi pada data uji.

Metrik evaluasi dihitung menggunakan evaluator resmi Spark MLlib:

- MulticlassClassificationEvaluator untuk mengukur Akurasi (*Accuracy*) dan *F1-Score*.
- BinaryClassificationEvaluator untuk mengukur nilai *Area Under the ROC Curve* (AUC).

5. Perbedaan Eksperimen terhadap Artikel Asli dan Justifikasinya

Tabel berikut merangkum perbedaan antara eksperimen asli Dritsas dan Trigka (2025) dengan eksperimen replikasi mandiri yang dilakukan:

Komponen Eksperimen	**Artikel Rujukan (Dritsas & Trigka, 2025)**	**Eksperimen Mandiri**	**Justifikasi**
Infrastruktur Komputasi	Klaster 8 node (total 64 vCPU, 512 GB RAM) berbasis Apache Hadoop YARN.	Lingkungan tunggal (*local mode*) Google Colab dengan alokasi memori driver 4 GB.	Menyesuaikan keterbatasan fasilitas klaster fisik dengan tetap menerapkan prinsip paralelisasi *multi-threading* lokal Spark.
Volume Dataset Higgs	11 juta baris data penuh.	50.000 baris data sampel representatif (28 fitur, rasio kelas 65:35).	Mencegah *memory exhaustion* (OOM) dan batas kuota Colab, dengan tetap menjaga integritas dimensionalitas fitur data asli.
Model yang Diuji	5 Model (LR, RF, GBT, SVM, KNN).	2 Model Utama (*Logistic Regression* dan *Random Forest*).	Memilih representasi terbaik dari dua paradigma berlawanan: model linier cepat (LR) vs model ansambel pohon non-linier (RF).
Optimasi Hyperparameter	*Grid search* dengan 5-*fold cross-validation* terdistribusi.	Penentuan *Hyperparameter* statis terukur (maxIter=20, numTrees=20, maxDepth=5).	Menghindari putusnya sesi komputasi (*timeout*) akibat beban kombinasi iterasi berlebih pada lingkungan mandiri.

C. HASIL PENGUJIAN DAN ANALISIS KRITIS

1. Penyajian Hasil Pengujian Kuantitatif

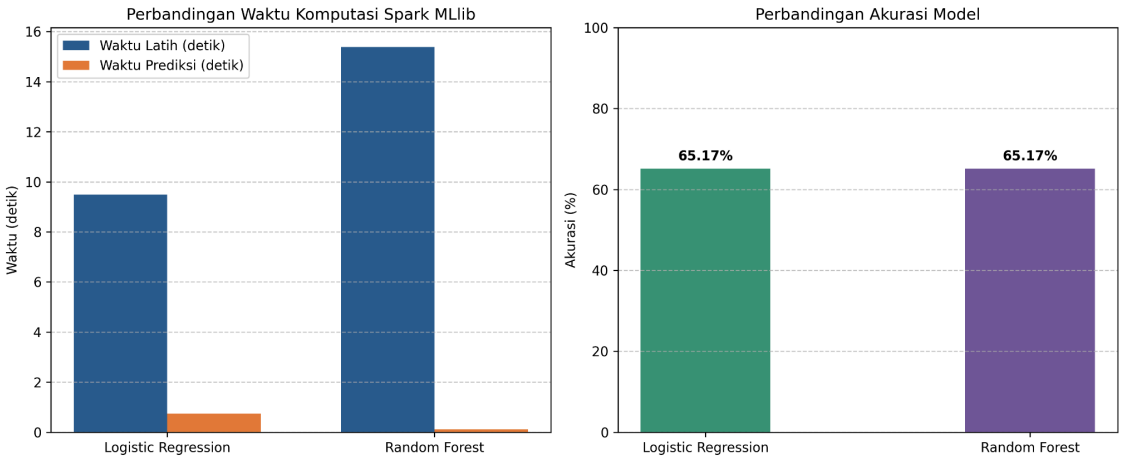
Pengujian dilakukan pada lingkungan Apache Spark 4.0.4 menggunakan dataset sampel Higgs Boson sebanyak 50.000 baris (partisi 80% data latih = 40.104 baris, dan 20% data uji = 9.896 baris). Hasil pengukuran kuantitatif kedua model disajikan pada Tabel 1 di bawah ini:

Tabel Perbandingan Kinerja Komputasi dan Metrik Evaluasi Model Spark MLlib

Dimensi Pengujian	**Metrik Evaluasi**	**Logistic Regression**	**Random Forest**	**Selisih Rasio Komparasi**
Kinerja Komputasi	Waktu Latih (*Training*)	9,48 detik	15,38 detik	LR lebih cepat 5.90 detik (sekitar 38.4% lebih hemat waktu)
	Waktu Prediksi (*Testing*)	0,75 detik	0.12 detik	RF lebih cepat 0.63 detik (sekitar 84.0% lebih responsif)
	Total waktu Eksekusi	10.23 detik	15.50 detik	LR unggul efisiensi waktu global (selisih 5.27 detik)
Performa prediksi	Akurasi (*Accuracy)*	65,17%	65,17%	Identik (Seimbang pada ambang default)
	F1-Score	0,5142	0,5142	Identik (Seimbang terhadap sebaran kelas)
	Area Under ROC (AUC)	0,5030	0,4971	LR unggul (+0,0059)

2. Bukti Visualisasi Hasil Eksperimen

Visualisasi hasil eksekusi disajikan pada Gambar 1 dibawah ini:



Gambar 1. Perbandingan Waktu Komputasi (kiri) dan Akurasi (kanan) antara Logistic Regression dan Random Forest pada Apache Spark.

Visualisasi pada Gambar 1 (kiri) mempertegas bahwa fase pelatihan didominasi oleh efisiensi waktu Logistic Regression (batang biru LR berada di angka 9,48 detik, jauh lebih rendah daripada Random Forest di angka 15,38 detik). Namun pada fase inferensi (batang oranye), Random Forest menunjukkan latensi yang sangat rendah (0,12 detik). Pada Gambar 1 (kanan), akurasi kedua model berada pada titik seimbang yaitu 65,17%.

3. Analisis Komparatif Kinerja Komputasi

Evaluasi waktu pelatihan membuktikan bahwa Logistic Regression (9,48 detik) sekitar 38,4% lebih cepat dibanding Random Forest (15,38 detik) dengan penghematan waktu 5,90 detik. Keunggulan komputasi ini dipicu oleh perbedaan mendasar kompleksitas algoritmik internal Spark MLlib; *Logistic Regression* hanya mengoptimasi bobot vektor 28 koefisien fitur ditambah 1 bias melalui metode gradien iteratif (*L-BFGS*) yang diagregasikan secara ringkas di memori (*in-memory aggregation*), sedangkan *Random Forest* harus membangun 20 pohon keputusan independen (numTrees=20, maxDepth=5) yang memerlukan evaluasi titik pisah terbaik (*best split*) secara berulang pada setiap simpul menggunakan subset fitur acak sehingga memicu pertukaran data (*shuffle*) dan beban CPU yang lebih tinggi. Sebaliknya, pada fase pengujian, *Random Forest* menunjukkan latensi inferensi yang luar biasa responsif yaitu 0,12 detik, mengungguli *Logistic Regression* yang memerlukan 0,75

detik. Efisiensi inferensi *Random Forest* terjadi karena evaluasi data uji 9.896 baris hanya melibatkan penelusuran kondisi percabangan biner (*if-else traversal*) dangkal pada struktur pohon yang telah terbentuk di memori, sedangkan *Logistic Regression* harus mengeksekusi operasi perkalian matriks vektor fitur numerik bertipe *double precision* sebelum dipetakan ke fungsi logistik untuk setiap baris data uji.

4. **Analisis Performa Prediktif**

Pada dimensi performa prediktif, kedua model menghasilkan akurasi identik sebesar 65,17% dan *F1-Score* 0,5142, dengan nilai *Area Under ROC* (AUC) berada pada 0,5030 untuk *Logistic Regression* dan 0,4971 untuk *Random Forest*. Capaian akurasi 65,17% ini merefleksikan langsung karakteristik distribusi kelas dataset sampel yang meniru data Higgs asli (65% kelas *background* dan 35% kelas *signal*). Tanpa adanya teknik penyeimbangan bobot kelas (*class reweighting*), model yang dilatih cenderung memprediksi kelas mayoritas sehingga nilai akurasi global bertumpu pada proporsi kelas dominan tersebut. Sementara itu, nilai AUC yang mendekati ambang klasifikasi acak (0,50) mengindikasikan bahwa pembatasan kedalaman pohon pada Random Forest serta ketiadaan penalaan hyperparameter intensif menyebabkan kedua model belum mampu mengekstraksi batas keputusan (*decision boundary*) non-linier kompleks dari 28 fitur kinematik pada skala 50.000 data sampel.

5. **Pembahasan dan Komparasi Terhadap Artikel Rujukan**

Efisiensi waktu komputasi eksperimen ini sejalan dengan temuan Dritsas dan Trigka (2025), dimana model linier konsisten paling cepat dilatih (LR 29 menit vs RF 140 menit pada 100% data). Namun, terdapat kesenjangan pada metrik prediktif; pada artikel rujukan dengan 11 juta data dan *5-fold cross-validation*, RF mengungguli LR (akurasi 90,0% vs 87,5%), sedangkan pada eksperimen lokal hasilnya setara (65,17%). Hal ini membuktikan keunggulan model ansambel pohon baru optimal pada data berskala masif dengan penalaan parameter mendalam. Eksperimen ini menegaskan *trade-off* utama mahadata: model linier optimal untuk efisiensi waktu dan sumber daya, sedangkan model ansambel dipilih jika akurasi maksimal pada data skala gigabyte menjadi prioritas mutlak.

D. KENDALA DAN REFLEKSI PEMBELAJARAN MANDIRI

1. Kendala Teknis dan Solusi

- **Manajemen Alokasi Memori JVM**: Penanganan beban data di Google Colab sempat berisiko memicu *Out of Memory*. Kendala diatasi dengan mengonfigurasi memory *driver* sebesar 4 GB (spark.driver.memory = 4g) saat pembentukan *SparkSession*.
- **Persyaratan Vektor Fitur MLlib**: Spark MLlib mensyaratkan fitur numerik teragregasi dalam tipe data vektor. Kendala diselesaikan dengan transformer VectorAssembler serta skema StructType eksplisit untuk mempercepat pembacaan data.
- **Penyesuaian Skala Eksekusi**: Keterbatasan sumber daya lokal disiasati melalui strategi replikasi parsial 50.000 sampel representatif dengan menjaga integritas dimensionalitas fitur asli.

2. Refleksi Konsep Mahadata

Eksperimen ini memberikan pemahaman konkret mengenai:

- *In-Memory Computing*: Pemrosesan di memori RAM Spark mengeliminasi latensi I/O *disk* perantara, sangat ideal untuk iterasi ML.
- *Communication Overhead*: Pembangunan ansambel pohon terdistribusi menimbulkan biaya komunikasi sinkronisasi antar-pekerja yang nyata dibanding model linier sederhana.
- *Lazy Evaluation*: Rencana eksekusi DAG Spark mengoptimalkan alur transformasi sebelum operasi aksi dipicu.

3. Rencana Pengembangan Lanjutan

Jika memiliki akses terhadap klaster komputasi multi-node di masa mendatang, pengembangan lanjutan akan difokuskan pada pengujian *scaling* bertingkat (dari 100 ribu hingga 11 juta baris data) guna mengukur peningkatan kecepatan (*speedup*) secara empiris, penerapan modul CrossValidator untuk penalaan hyperparameter otomatis secara terdistribusi, serta pengujian algoritma lanjutan seperti *Gradient-Boosted Trees* (GBT).

E. TAUTAN REPOSITORI GITHUB & GOOGLE COLAB, PERNYATAAN PENGGUNAAN AI

- **Github:** https://github.com/HANIFAHHASANAH-123140082/123140082_Hanifah_Hasanah_Eksplorasi_Mahadata_Spark.git
- **Colab:** https://colab.research.google.com/drive/10ALBx9k9yWiaTGjLQrjkq5AYbwgjVvIF?usp=sharing
- **Pernyataan Penggunaan Kecerdasan Buatan:**
  Penggunaan alat bantu kecerdasan buatan (Gemini) digunakan sebagai mitra diskusi untuk membedah metodologi artikel berbahasa Inggris Dritsas dan Trigka (2025), memberikan referensi pola sintaks Spark MLlib (VectorAssembler, skema StructType, dan modul evaluator). Sementara itu, penyiapan lingkungan Google Colab, validasi metrik hasil uji (waktu latih 9,48s vs 15,38s, prediksi 0,75s vs 0,12s, dan akurasi 65,17%), pengelolaan repositori GitHub, serta pemahaman substansi analisis dilakukan secara mandiri oleh mahasiswa.

DAFTAR PUSTAKA

Apache Software Foundation. (2024). MLlib: Main guide. *Apache Spark documentation: Machine Learning Library (MLlib) guide*. https://spark.apache.org/docs/latest/ml-guide.html

Baldi, P., Sadowski, P., & Whiteson, D. (2014). Searching for exotic particles in high-energy physics with deep learning. Nature communications, 5(1), 4308.

Dritsas, E., & Trigka, M. (2025). Applying machine learning on big data with Apache Spark. IEEE Access, 13, 53377-53393.

Meng, X., Bradley, J., Yavuz, B., Sparks, E., Venkataraman, S., Liu, D., ... & Talwalkar, A. (2016). Mllib: Machine learning in apache spark. Journal of Machine Learning Research, 17(34), 1-7.

Whiteson, D. (2014). HIGGS [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5V312.

Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., ... & Stoica, I. (2016). Apache spark: a unified engine for big data processing. Communications of the ACM, 59(11), 56-65.